

2. Structured CSV as a 21st-Century Normalized Table: LHM, Syntax Binding, and Semantic Binding

This document explains the shared UADC data layer. Structured CSV is not one denormalized relation. It is a physical container for multiple logically normalized row scopes, connected by sparse ownership and dimension columns.

Binding Model Overview

Syntax Binding

Syntax binding maps a physical syntax such as XML to the LHM. In Phase 1, UBL **XPath**, attributes, conditions, and **selectors** are bound to LHM classes and attributes. Forward processing extracts values; reverse processing reconstructs XML from the same binding.

```
UBL XPath / selector / attribute
      ↑↓
    LHM semantic_path
      ↑↓
    Structured CSV column
```

Semantic Binding

Semantic binding does not return to the source XML. It maps an LHM **semantic_path** and Structured CSV column to a purpose-specific target column. The source column is derived from the final semantic-path segment, while class multiplicity determines row scope.

A zero-based **[n]** selects one occurrence when a repeated class must be expanded horizontally:

```
$.invoice.vatBreakdown[0].vatCategoryCode -> Tax1_Type_Code
$.invoice.vatBreakdown[1].vatCategoryCode -> Tax2_Type_Code
```

[n] selects an occurrence; it does not create an output row. Output row creation is controlled by non-indexed semantic paths and the deepest repeated class used by the target binding.

The current **semantic_binding.py** implementation interprets explicit zero-based **[n]** occurrence selectors. It does not evaluate XPath predicates. A business condition that cannot be represented by **[n]** must be resolved into a semantic column, default, transformation rule, or a future extension of the semantic-binding processor.

Authority of Each Definition

- The LHM is authoritative for meaning, hierarchy, multiplicity, and data type.

- A syntax binding is authoritative for XPath, selectors, attributes, and syntax-specific conditions.
- A semantic binding is authoritative for target columns, order, **semantic_path**, **[n]**, and mapping status.
- Structured CSV is the stable syntax-neutral interface between Phase 1 and Phase 2.

Detailed Model and Supporting Tools

1. Purpose and Architecture

This document defines the shared data model used by UADC Phase 1 and Phase 2. It explains how the EN 16931 Logical Hierarchical Model (LHM), syntax-binding tables, Structured CSV rows, xBRL-CSV metadata and taxonomy, and semantic-binding tables fit together. It also specifies the supporting programs under **tools/** at a consistent level of detail.

The architecture separates three concerns:

- the LHM assigns stable semantic identifiers, hierarchy, cardinality, and data types;
- a syntax binding maps one source syntax, such as UBL Invoice, to the LHM;
- a semantic binding maps LHM values to a Phase 2 target, such as ADS PSV, ADS XBRL GL, or ISO 21378 ADC.

Structured CSV is the durable interface between Phase 1 and Phase 2. It retains the semantic hierarchy without making downstream systems depend on the source XML syntax.

1.1 Hierarchical Tidy Data

Hierarchical Tidy Data extends the tidy-data principle to information whose natural structure contains nested and repeatable classes. Each semantic attribute still has one defined column, each occurrence is represented once, and each cell contains one value. In addition, dimension columns state the class occurrence to which each sparse row belongs.

A conventional flat table repeats invoice-header and party values on every invoice-line row. That repetition makes the file easy to query as one rectangle, but it duplicates data and can create inconsistent copies of the same fact. Hierarchical Tidy Data instead emits the invoice, party, allowance, tax, and line occurrences at their own row scopes. Ancestor dimension values connect those rows without copying all ancestor attributes into every descendant row.

The resulting table is hierarchical in meaning but tidy in storage:

- every semantic fact has one column determined by the LHM;
- every class occurrence has one owning row scope;
- singular child classes share the nearest repeated ancestor row;
- repeated child classes use independent rows from their first occurrence;
- ancestor and child dimensions identify the complete hierarchical context;
- blank cells outside a row's scope mean **not owned by this row**, rather than another copy of a null business fact.

1.2 Structured CSV as an Alternative Normalization Form

For semantic interchange, Structured CSV is proposed as an alternative to **Codd's relational normalization**. Codd-style normalization removes update anomalies by decomposing data into multiple relations, storing each fact once, and connecting the relations through primary and foreign keys. This is effective inside a relational database, but an interchange package must then carry and coordinate several tables, their keys, and their join rules.

Structured CSV keeps the same essential objective—one authoritative occurrence of each fact without redundant copies—but represents the normalized relations in one physical table. It combines what would otherwise be several normalized tables while preserving their logical separation through sparse row ownership and dimension columns.

Relational normalization	Hierarchical Structured CSV
One relation for each entity or repeating group	One row scope for each repeated LHM class
Primary key identifies a row in one relation	Dimension value identifies a class occurrence
Foreign key relates a child relation to its parent	Ancestor dimension columns relate a child row to its parent row
Join combines several physical tables	Matching dimension paths reconstruct the hierarchy within one CSV
Parent values are not copied to child relations	Parent attributes remain on the parent row and are not copied to child rows

Thus, **one CSV file does not mean one denormalized relation**. It is a container for several logically normalized row scopes. The dimensions provide the relationships that separate relational tables would otherwise express with foreign keys. A consumer can reconstruct individual logical tables by selecting rows owned by each dimension, or reconstruct the complete hierarchy by grouping rows on their ancestor-dimension paths.

This model does not invalidate relational normal forms or database constraints. It provides a different normalization and serialization mechanism for durable, syntax-neutral data exchange: multiple logical tables are combined into one file without duplicating their facts, and their relationships remain explicit.

1.3 Normalization Invariants

A Structured CSV conforms to this normalization model when all of the following conditions hold:

- 1. a semantic value is written only in the row scope that owns its LHM class;
- 2. a parent value is not repeated merely because a child class repeats;
- 3. a repeated child's first occurrence is not merged into the parent row;
- 4. every child row carries the ancestor dimensions required to identify its parent occurrence;

5. dimension values are unique within their ancestor scope;
6. the original hierarchy can be reconstructed without depending on physical row adjacency alone;
7. syntax-binding and semantic-binding processing preserve these ownership and dimension rules in both directions.

Chapter 4 gives the normative singular-child and repeated-child row patterns. The validation functions described in Chapters 8 and 15 enforce the same invariants during Phase 1 and Phase 2 processing.

2. EN 16931 LHM

2.1 Source and Generated Files

The LHM source and generated artifacts are maintained under **data/** and **out/phase1/**. The authoritative filenames and generation commands are listed in **01_ENVIRONMENT_TESTS_TUTORIAL.md**. Generation programs under **tools/** normalize source tables, derive semantic paths, and create files consumed by converters and taxonomy generators.

2.2 Core Columns

The model uses the following information across definition and binding tables:

- semantic identifier for a business term or class;
- type, where **C** identifies a class and **A** identifies an attribute;
- parent or level information defining the class hierarchy;
- multiplicity defining whether a class is singular or repeatable;
- semantic path identifying a value independently of a source syntax;
- syntax-specific XPath, selector, data type, and conversion conditions.

The combination of identifier, hierarchy, and multiplicity is authoritative. Indentation, row order, and labels improve readability but do not replace these structural columns.

2.3 Effective Hierarchy

Class rows form the hierarchy. Attribute rows belong to the nearest applicable class. A singular child class is folded into the current row scope of its nearest repeated ancestor. A repeated child class creates its own row scope from its first occurrence. This rule is shared by binding generation, Structured CSV creation, and reverse conversion.

2.4 Naming and Stability

Semantic identifiers and established namespace identifiers remain stable across syntax revisions. Human-readable labels may be improved without changing data identity. Generated taxonomy entry points use a dated filename convention, including **en16931-oim-2026-07-05.xsd**.

3. Semantic Path, XPath, and Binding Authority

3.1 Semantic Path

A semantic path identifies a class or business term in the LHM hierarchy. It is used to determine the source Structured CSV column and repeated row scope for Phase 2 conversion. Phase 2 converters therefore do not reconstruct source UBL XPath rules.

3.2 Syntax XPath

The syntax-binding table maps semantic paths to source or target XPath. Forward conversion evaluates source XPath against the XML document. Reverse conversion constructs the absolute XPath from its document root and creates missing ancestors in document order. Absolute processing prevents an element such as **AccountingSupplierParty** from being created under a nested **Invoice** found inside **PaymentMeans**.

3.3 Binding Authority

The applicable binding table is authoritative for syntax-specific selectors, attributes, conditions, and occurrence handling. The LHM remains authoritative for semantic hierarchy and multiplicity. Generated join tables must preserve both authorities and expose conflicts during validation instead of silently choosing one.

4. Structured CSV Hierarchy

4.1 Dimension Columns

Structured CSV represents each repeated class with a dimension column. A dimension value identifies an occurrence within the current ancestor scope. Attribute columns hold values; blank cells mean that the row does not own that attribute.

4.2 Singular Child Class

If **dBbb** is singular under repeated **dAaa**, values of both classes share the same row:

```
dAaa, dBbb, a1, a2, b1, b2, b3
1, , a1V1, a2V1, b1V1, b2V1, b3V1
```

The singular child belongs to the nearest repeated ancestor row and does not receive an independent occurrence row.

4.3 Repeated Child Class

If **dBbb** is repeatable, the parent attributes occupy the parent row and every child occurrence occupies an independent row, including the first occurrence:

```
dAaa,dBbb,a1,a2,b1,b2,b3
1,,a1V1,a2V1,,,
1,1,,,b1V1,b2V1,b3V1
1,2,,,b1V2,b2V2,b3V2
```

The following compact form is invalid because it merges the first repeated child into the parent row:

```
dAaa,dBbb,a1,a2,b1,b2,b3
1,1,a1V1,a2V1,b1V1,b2V1,b3V1
1,2,,,b1V2,b2V2,b3V2
```

4.4 Ownership Derivation

Converters derive column ownership from binding rows where **type=C**, the class **multiplicity**, and each attribute **semantic_path**. A singular child inherits the row scope of the nearest repeated ancestor. A repeated child owns a new dimension and emits an independent row from occurrence 1. This rule is applied recursively at every depth.

4.5 Sparse Row Contract

Rows are intentionally sparse. A parent row contains only values owned by its scope; a repeated-child row contains the dimensions needed to identify its ancestry and the values owned by that child scope. Phase 2 merging carries current ancestor values into a target record only where the target binding requires them.

5. xBRL-CSV Taxonomy Model

5.1 Entry Point and Definition Linkbase

The OIM entry point is **en16931-oim-2026-07-05.xsd**. Its definition linkbase is **en16931-def-2026-07-05.xml**. The dated names distinguish this generated taxonomy set from earlier tuple-oriented or non-OIM entry points.

5.2 Taxonomy Components

The OIM taxonomy contains:

- an entry-point schema importing the concept and dimension schemas;
- concepts for LHM business terms;
- explicit dimensions representing repeated class occurrences;
- domain and member declarations required by xBRL-CSV contexts;
- a definition linkbase connecting hypercubes, dimensions, domains, and applicable concepts;
- labels and references when supplied by the source LHM.

No tuple support is required for the UADC OIM taxonomy. Hierarchy is expressed through dimensions and definition relationships.

5.3 Dimension Derivation

Each repeatable LHM class becomes a dimension scope. Singular descendants remain within that scope. Nested repeatable classes add dimensions while retaining their ancestor dimensions. This mirrors the Structured CSV ownership rules in Chapter 4 and allows Arelle to expose the intended dimensional relationships.

6. Currencies, Selectors, and Conditional Facts

6.1 Currency Codes

Currency identifiers are normalized through the applicable code-list table, including **Currency.csv**. Amount columns retain their numeric value while the binding or xBRL-CSV metadata supplies the corresponding currency unit.

6.2 BT-110 and BT-111

BT-110 Invoice total VAT amount is selected when the amount currency code equals the document currency code. **BT-111 Invoice total VAT amount in accounting currency** is selected when it equals the tax accounting currency code. The condition uses the amount element's currency identifier together with the document-level currency fields; it must not depend only on element order.

For the Allowance example, Structured CSV must retain the two conditional values in separate semantic columns: document-currency VAT total **1225.00** and accounting-currency VAT total **9324.00**. Blank conditional facts indicate a binding, selector, or currency-comparison defect and are covered by regression tests.

6.3 Seller Address Selector

In the ADS binding, seller identity is selected through identifier type **V**. The associated address is below that selector through **identifierAddress**. Join-table generation must retain the selector context so seller address fields appear in the Phase 2 ADS XBRL GL seller list.

7. Audit Continuity and Data Boundaries

ADS and ADC normally operate in an ERP environment where user activity, accounting detail, closing entries, and annual-report preparation are recorded outside an individual invoice. Consequently, an invoice alone cannot populate every audit-trail, posting, approval, or reporting field in those target models.

UADC makes that boundary explicit. It maps all information supported by the invoice, preserves missing target fields as known gaps, and retains stable semantic identifiers and hierarchy between independently designed interface files. An organization may therefore keep its own input and

output formats while using the LHM and binding mechanism as a durable semantic layer. Together with ERP operation history and controlled binding versions, this provides a basis for long-term, reproducible, audit-ready data retention.

8. Supporting Tool Specification

8.1 Purpose

This document specifies every supporting program under **tools/**. It records the role, inputs, outputs, command-line interface, processing sequence, major functions and data structures, validation behavior, and operational constraints of each tool at a consistent level of detail.

The tools support four activities:

- EN 16931 LHM construction and maintenance;
- syntax-binding preparation and tutorial conversion;
- taxonomy, metadata, and round-trip artifact generation;
- inspection of generated flat files.

Operational Phase 1 and Phase 2 converters remain under **src/**. Programs under **tools/** either prepare their definitions and test artifacts or provide smaller tutorial implementations.

8.2 Scope

The specification covers these 15 programs:

No.	Program	Classification
1	audit_en16931_coverage.py	LHM audit
2	build_lhm_from_source.py	LHM generation
3	build_roundtrip_test_artifacts.py	Test-artifact generation
4	build_syntax_binding.py	Binding generation
5	check_lhm_class_element.py	LHM validation
6	extend_en16931_lhm_coverage.py	LHM maintenance
7	normalize_lhm_class_element.py	LHM normalization
8	normalize_lhm_semantic_paths.py	LHM normalization
9	order_lhm_by_en16931_table2.py	LHM ordering
10	psv_viewer.html	Browser inspection
11	tutorial/semantic_binding_sample.py	Tutorial semantic conversion
12	tutorial/syntax_binding_sample.py	Tutorial syntax conversion
13	taxonomy/xBRLGL_TaxonomyGenerator.py	Taxonomy generation

No.	Program	Classification
14	update_lhm_definitions_from_pdf.py	LHM definition update
15	update_lhm_syntax_sequence_from_ubl_xsd.py	LHM syntax-order update

8.3 Shared Conventions

8.3.1 Execution Directory

Commands in this document are run from the UADC PoC repository root.

Windows PowerShell:

```
$python = 'python'
& $python .\tools\PROGRAM.py [arguments]
```

macOS or Linux:

```
PYTHON=python3
$PYTHON ./tools/PROGRAM.py [arguments]
```

8.3.2 CSV Encoding

Tools with an encoding option default to **utf-8-sig**. This preserves a UTF-8 byte-order mark for compatibility with spreadsheet software. Tools without an encoding option also read and write LHM CSV files as **utf-8-sig**.

8.3.3 Exit Status

The normal convention is:

- exit status **0**: processing or validation succeeded;
- exit status **1**: a handled validation failure or conversion error occurred;
- uncaught exception: an input file, schema, required column, or dependency is missing or invalid.

8.3.4 In-place Modification

The following programs modify their input LHM CSV directly:

- **extend_en16931_lhm_coverage.py**;
- **normalize_lhm_class_element.py**;
- **order_lhm_by_en16931_table2.py**;
- **update_lhm_definitions_from_pdf.py**.

normalize_lhm_semantic_paths.py also modifies the input in place when **--output** is omitted. Review or copy the source before running these tools.

8.4 Tool Specifications

8.4.1 audit_en16931_coverage.py

Role

Audits whether every EN 16931 **BG-n** and **BT-n** identifier found in the standard PDF is represented in the LHM CSV.

Inputs and Outputs

- Input 1: EN 16931-1 PDF.
- Input 2: LHM CSV containing an **id** column.
- Output: a JSON report written to standard output.
- Side effects: none.

Command

```
& $python .\tools\audit_en16931_coverage.py STANDARD.pdf LHM.csv
```

Processing

1. **extract_pdf_ids** reads every PDF page with **pypdf.PdfReader**.
2. The regular expression **BG-n** or **BT-n** collects the first page on which each identifier appears.
3. **read_lhm_ids** collects matching identifiers from the LHM **id** column.
4. The program calculates missing and extra identifier sets.
5. It prints counts, missing identifiers with PDF page numbers, and extra LHM identifiers as formatted JSON.

Functions

Function	Responsibility
sort_key	Sorts identifiers by BG or BT prefix and numeric suffix.
extract_pdf_ids	Extracts identifier-to-first-page mappings from the PDF.
read_lhm_ids	Reads valid BG and BT identifiers from the LHM.
main	Parses inputs, compares sets, prints JSON, and returns status.

Exit and Dependencies

Returns **1** when at least one PDF identifier is missing from the LHM; otherwise returns **0**. Requires **pypdf**. Extra LHM identifiers are reported but do not by themselves cause failure.

8.4.2 build_lhm_from_source.py

Role

Maintains a separation between an editable EN 16931 source table and the normalized LHM CSV consumed by conversion and taxonomy scripts.

Inputs and Outputs

The **init-source** command reads a generated LHM and writes an editable source CSV. The **build** command reads that editable source and writes a generated LHM with normalized types, multiplicities, hierarchy levels, semantic paths, class terms, and unique element names.

Commands

```
& $python .\tools\build_lhm_from_source.py init-source LHM.csv SOURCE.csv  
& $python .\tools\build_lhm_from_source.py build SOURCE.csv LHM.csv
```

Processing

For **init-source**, **source_from_lhm** copies reviewable business fields and creates override columns. For **build**:

1. Source rows are sorted by **sequence**.
2. **type_from_id** derives class or attribute type when type is absent.
3. **normalize_multiplicity** converts source cardinality to the supported LHM representation.
4. **semantic_path** uses an explicit override or assembles lower-camel-case segments following **path** identifiers.
5. **nearest_bg_id** selects the semantic class for each fact.
6. **lhm_effective_levels** assigns levels only to the invoice root, repeated classes, and facts owned by the nearest effective repeated ancestor.
7. Explicit class-term and element overrides are retained; missing elements are generated by **unique_element_names**.
8. Duplicate elements and unsupported multiplicities are rejected before the LHM is written.

Functions

Function	Responsibility
normalize_multiplicity	Normalizes cardinality notation.
read_rows, write_rows	Read and write UTF-8 BOM CSV records.
type_from_id	Maps BG to class and BT to attribute rows.
lower_camel_case_concatenated	Creates semantic path segments.
source_from_lhm	Creates the editable source form.
path_ids	Returns the hierarchy identifiers for one row.
semantic_path	Builds or applies the semantic path.
nearest_bg_id	Finds the owning business group.
lhm_effective_levels	Calculates effective hierarchical CSV levels.
build_lhm	Orchestrates generated LHM construction.
validate_unique_elements	Rejects duplicate concept names.
validate_multiplicities	Rejects unsupported multiplicities.
main	Dispatches init-source or build .

Validation

After generation, run the LHM semantic-path, hierarchical-layout, and taxonomy generator regression tests. The generated LHM is authoritative; manual changes should normally be made in the editable source CSV or override columns.

8.4.3 build_roundtrip_test_artifacts.py

Role

Rebuilds the complete Phase 1 round-trip artifact sets used by regression tests.

Inputs and Outputs

Inputs are fixed by **DATASETS**:

- the minimal OpenPeppol UBL Invoice sample;
- all UBL Invoice XML files under the BIS Billing 3 sample directory.

For each dataset it recreates:

```
tests/roundtrip/DATASET/source_xml/
tests/roundtrip/DATASET/structured_csv/
```

```
tests/roundtrip/DATASET/metadata_json/  
tests/roundtrip/DATASET/roundtrip_xml/
```

Command

```
& $python .\tools\build_roundtrip_test_artifacts.py
```

The program has no command-line options. Supplying **--help** still runs the builder because the script does not use an argument parser.

Processing

1. **ensure_taxonomy** verifies the OIM entry schema and EN 16931 module schema; if absent, it invokes the taxonomy generator regression script.
2. **is_invoice_xml** excludes XML documents whose root element is not Invoice.
3. **clean_directory** removes existing files from each target artifact folder.
4. **build_dataset** copies source XML, generates Structured CSV and metadata JSON, then reverse-converts the CSV into UBL XML.
5. **run_converter** invokes **src/syntax_binding.py** through the current Python executable and fails immediately when the child process fails.

Functions

Function	Responsibility
ensure_taxonomy	Ensures required generated taxonomy schemas exist.
is_invoice_xml	Checks the XML root local name.
clean_directory	Recreates and empties one artifact directory.
run_converter	Runs forward or reverse syntax binding.
build_dataset	Builds all four artifacts for one sample collection.
main	Builds both configured datasets and reports the total.

Constraints

This is a regenerating tool. Existing files directly inside the four target artifact folders are deleted before rebuilding. Run the round-trip artifact, UBL schema, and xBRL-CSV metadata validation scripts after generation.

8.4.4 build_syntax_binding.py

Role

Converts a simple table of **semantic_path** and **xpath** pairs into either a compact binding CSV or an LHM/HMD-style syntax binding.

Inputs and Outputs

- Input: CSV containing both **semantic_path** and **xpath**.
- Output: binding CSV selected by **--output**.
- Default encoding: **utf-8-sig**.

Command

```
& $python .\tools\build_syntax_binding.py BINDINGS.csv `
  --output SYNTAX_BINDING.csv [--simple] [--encoding utf-8-sig]
```

Without **--simple**, the output contains generated class rows and attribute rows in the full LHM header. With **--simple**, the output contains only **column**, **semantic_path**, and **xpath**.

Processing

1. **read_source_rows** validates required columns and rejects incomplete pairs.
2. **split_semantic_path** splits on dots outside predicate brackets.
3. **clean_token** and **semantic_path_to_column** create safe stable columns.
4. **unique_name** adds numeric suffixes to collisions.
5. **build_simple_rows** emits one row per input pair, or **build_lhm_rows** creates missing ancestor class rows followed by attribute rows.
6. **build_syntax_bindings** creates the destination directory and writes rows.

Functions

Function	Responsibility
semantic_path_to_column	Derives a target column from a semantic path.
split_semantic_path	Preserves predicates while splitting path segments.
segment_label, segment_element	Derive labels and element tokens.
clean_token, unique_name	Sanitize and deduplicate identifiers.
read_source_rows	Validates source pairs.
build_simple_rows	Produces the compact format.

Function	Responsibility
build_lhm_rows	Produces generated class and attribute rows.
build_syntax_bindings	Writes the selected output format.
main	Parses arguments and converts handled exceptions to status 1 .

Constraints

Generated class multiplicity is **1..***, generated fact multiplicity is **0..1**, and generated fact datatype is Text. Review these defaults before using the result as a production binding.

8.4.5 check_lhm_class_element.py

Role

Validates that LHM **class_term** and **element** values match the normalization rules without modifying the CSV.

Command and Input

```
& $python .\tools\check_lhm_class_element.py LHM.csv
```

Processing

1. Reads the LHM and maps identifiers to names.
2. Reuses **nearest_parent_bg**, **singularize**, and **unique_element_names** from **normalize_lhm_class_element.py**.
3. Compares expected and stored values for BG and BT rows.
4. Prints up to 50 mismatches in each category and a total count.

Functions

Function	Responsibility
read_rows	Reads normalized LHM dictionaries.
main	Calculates expected values, reports differences, and returns status.

Returns **1** if either mismatch list is non-empty; otherwise prints the checked row count and returns **0**.

8.4.6 extend_en16931_lhm_coverage.py

Role

Applies the program's curated EN 16931 coverage additions and corrections to an existing LHM CSV.

Command and Side Effect

```
& $python .\tools\extend_en16931_lhm_coverage.py LHM.csv
```

The input file is rewritten in place.

Processing

- 1. Static **UPDATES** amend known rows by identifier.
- 2. Static **ROWS** define missing BG and BT records.
- 3. Existing identifiers are not duplicated.
- 4. **write_rows** rennumbers **sequence** and recalculates **level** from **path**.

Functions and Data

Item	Responsibility
class_row, attr_row, row	Build normalized curated records.
UPDATES	Field corrections for existing identifiers.
ROWS	Curated records to add when absent.
read_rows, write_rows	Read and rewrite the full LHM.
main	Applies updates and reports the number of additions.

Constraints

The program is data-driven but not a general PDF parser. After execution, run the ordering, semantic-path normalization, class/element normalization, and LHM checks so derived fields remain consistent.

8.4.7 normalize_lhm_class_element.py

Role

Normalizes LHM **class_term** and **element** values in place.

Command

```
& $python .\tools\normalize_lhm_class_element.py LHM.csv
```

Rules

- A BG row belongs to its own group; a BT row belongs to its nearest BG ancestor.
- **class_term** is the singularized owning BG business term.
- **element** starts with an uppercase character and is derived from the shortest unique suffix of **semantic_path**.
- Duplicate names are expanded by adding more semantic path segments.

Functions

Function	Responsibility
singularize	Applies the English singularization rules.
semantic_segments	Splits a semantic path into name segments.
upper_camel	Converts one segment to UpperCamelCase.
suffix_element_name	Builds an element from a path suffix.
unique_element_names	Finds the shortest unique name for every row.
nearest_parent_bg	Finds the owning BG identifier.
read_rows, write_rows	Read and rewrite the input CSV.
main	Updates changed cells and reports the value count.

Run **check_lhm_class_element.py** immediately after normalization.

8.4.8 normalize_lhm_semantic_paths.py

Role

Rebuilds every LHM **semantic_path** from row names and the identifier hierarchy.

Command

```
& $python .\tools\normalize_lhm_semantic_paths.py INPUT.csv `
  [--output OUTPUT.csv] [--encoding utf-8-sig]
```

When **--output** is omitted, **INPUT.csv** is replaced.

Processing

1. **lower_camel_case_concatenated** converts each business term to one path segment, using **unnamed** for empty terms and prefixing numeric starts.
2. **normalize_rows** maps row identifiers to segments.
3. For each row, segments are selected in **path** order.
4. The resulting value is written as **\$.segment.child**.
5. **normalize_file** retains the original CSV column order.

Functions

Function	Responsibility
lower_camel_case_concatenated	Creates lower-camel-case segments.
normalize_rows	Replaces semantic paths in memory.
normalize_file	Reads, normalizes, and writes a complete CSV.
main	Selects output location and handles errors.

Explicit semantic-path overrides are not consulted by this standalone tool. Use **build_lhm_from_source.py** when editable source overrides must be retained.

8.4.9 order_lhm_by_en16931_table2.py

Role

Orders all LHM rows according to the embedded EN 16931-1 Table 2 identifier sequence and applies known hierarchy corrections.

Command and Side Effect

```
& $python .\tools\order_lhm_by_en16931_table2.py LHM.csv
```

The input file is rewritten in place.

Processing

1. **FIXES** updates known row fields before sorting.
2. **TABLE2_ORDER** supplies the complete identifier order beginning with the invoice root.
3. Any identifier absent from the order list raises an error; unknown rows are never silently placed at the end.
4. Rows are sorted, **sequence** is renumbered, and **level** is recalculated from **path**.

Functions and Data

Item	Responsibility
TABLE2_ORDER	Defines normative program ordering.
FIXES	Applies curated hierarchy or field corrections.
read_rows, write_rows	Read and rewrite ordered records.
main	Validates completeness, sorts, and reports the row count.

8.4.10 psv_viewer.html**Role**

Provides a standalone browser viewer for generated ADS PSV, CSV, tab-delimited, or text files. It does not upload data and requires no web server.

User Inputs and Outputs

- Input: local file selected or dropped into the page.
- Delimiter: pipe, comma, or tab.
- Filter: case-insensitive text applied to rendered rows.
- Output: an interactive HTML table in the browser only.

Processing

1. **loadFile** reads the selected file with **FileReader** and removes a BOM.
2. **normalizedDelimiter** maps the UI choice to the actual delimiter character.
3. **parseDelimited** parses quoted cells, escaped quotes, delimiters, and line endings without a server-side library.
4. **renderTable** treats the first row as headers, adds row numbers, and hides columns empty in every data row.
5. **applyFilter** toggles rows based on searchable lower-case row text.
6. **updateMeta** displays file name, row count, column count, hidden columns, and visible-row count.

Functions

Function	Responsibility
parseDelimited	Parses delimiter-separated text with quote handling.
normalizedDelimiter	Resolves pipe, comma, or tab.
renderMessage	Displays empty-file or warning messages.

Function	Responsibility
renderTable	Builds the sticky-header HTML table.
updateMeta	Updates file and table statistics.
applyFilter	Filters rows without reparsing the file.
loadFile	Coordinates local file reading and rendering.

Constraints

The viewer is for inspection, not validation or editing. All processing occurs in browser memory. Closing or refreshing the page discards the loaded data.

8.4.11 tutorial/semantic_binding_sample.py

Role

Demonstrates a simple row-for-row Structured-CSV-to-flat-CSV semantic binding. It is not the operational Phase 2 converter. Production processing uses **src/semantic_binding.py**, which understands class multiplicity, repeated row scope, indexed semantic paths, target formats, and directory input.

Command

```
& $python .\tools\tutorial\semantic_binding_sample.py STRUCTURED.csv `
  --binding BINDING.csv --output FLAT.csv [--encoding utf-8-sig]
```

Binding Compatibility

The reader accepts legacy alternatives for target name, source path, and fixed value columns. The final source segment after a slash or dot is used as the Structured CSV column name.

Processing

1. **read_bindings** selects rows having a target and either source or fixed value.
2. **tail** reduces the source path to a column name.
3. The input Structured CSV is read as independent rows.
4. For every input row, each target receives its source cell or fixed fallback.
5. One output row is written for every input row.

Functions

Function	Responsibility
first_present	Reads the first populated legacy field name.
tail	Extracts the final path segment.
read_bindings	Normalizes usable binding rows.
convert_structured_to_flat	Performs row-for-row projection.
main	Parses arguments, reports dimensions, and handles errors.

Constraints

This tutorial does not merge hierarchical rows, infer repeated ancestors, or enforce the single-child versus repeated-child Structured CSV rules.

8.4.12 tutorial/syntax_binding_sample.py

Role

Demonstrates a compact XML-to-CSV syntax-binding implementation. It is not the operational Phase 1 converter. Production conversion and reverse conversion use **src/syntax_binding.py**.

Command

```
& $python .\tools\tutorial\syntax_binding_sample.py INPUT.xml `
  --binding BINDING.csv --output OUTPUT.csv `
  [--row-xpath XPATH] [--encoding utf-8-sig]
```

Processing

1. **collect_namespaces** reads namespace declarations from the source XML.
2. **read_bindings** accepts several legacy binding column names.
3. **find_nodes** implements a limited child-step XPath evaluator.
4. Optional **--row-xpath** chooses repeated row contexts; without it the document root produces one row.
5. Each binding extracts an element text or terminal attribute; multiple text matches are pipe-joined.
6. A binding default is used when extraction returns an empty value.

Supported Predicates

The limited evaluator supports:

- child-path equality or inequality with a quoted literal;
- equality or inequality with an absolute document path;
- child-path comparison with **true()** or **false()**;
- terminal attributes and namespace-prefixed element steps.

Unsupported predicates are not rejected and currently behave as matches. This is an important reason not to use the sample converter as the runtime authority.

Functions

Function group	Responsibility
local_name, collect_namespaces, qualify_step	Namespace handling.
split_step_predicate, split_xpath, split_terminal_attribute	XPath parsing.
path_value, predicate_matches, child_matches	Predicate evaluation.
find_nodes, get_value	XML node and scalar extraction.
first_present, read_bindings	Legacy binding normalization.
write_structured_csv	Emits one row per selected context.
main	Parses arguments and handles conversion errors.

8.4.13 taxonomy/xBRLGL_TaxonomyGenerator.py

Role

Generates the local XBRL-CSV taxonomy, dimensional definition relationships, presentation relationships, labels, metadata template, and CSV skeleton from an LHM/HMD-style definition CSV.

Command

```
& $python .\tools\taxonomy\xBRLGL_TaxonomyGenerator.py LHM.csv `
--base_dir out\taxonomy `
--palette en16931 `
--root invoice `
--lang en `
--currency JPY `
--namespace http://www.xbrl.org/int/gl/plt/2026-07-05 `
[--encoding utf-8-sig] [--trace] [--debug]
```

Arguments

Argument	Meaning
inFile	Input LHM/HMD CSV.
--base_dir	Generated taxonomy root; default is the current directory.
--palette	Default module when a record has no module.
--root	Root semantic identifier used for metadata columns.
--lang	Local label language; default ja .
--currency	ISO 4217 unit for amount columns; default JPY .
--namespace	Taxonomy namespace; final ten characters define the version.
--encoding	CSV and generated text encoding.
--trace, --debug	Diagnostic output controls.

Processing

1. The constructor validates the input and creates the output root.
2. **load_csv_data** accepts the UADC LHM header, normalizes records, maps datatypes, derives presentation parents, and builds effective dimension ownership from **lhm_level** and multiplicity.
3. Only facts and effective dimension classes are retained for taxonomy output.
4. **process_records** derives repeated-class parent relationships and role data.
5. **generate_taxonomy_files** writes module schemas, OIM entry schema, presentation, labels, and dimensional definition relationships.
6. **ensure_gl_gen_schema** locates or versions the generic datatype schema.
7. **json_meta_file** writes an xBRL-CSV JSON template and CSV skeleton when a root is supplied.

Primary Generated Files

For the EN 16931 profile and version **2026-07-05**, the important files are:

```

out/taxonomy/en16931/en16931-2026-07-05.xsd
out/taxonomy/gen/gl-gen-2026-07-05.xsd
out/taxonomy/plt/en16931-oim-2026-07-05.xsd
out/taxonomy/plt/en16931-def-2026-07-05.xml
out/taxonomy/en16931/en16931-2026-07-05-presentation.xml
out/taxonomy/xbml-gl.json
out/taxonomy/xbml-gl_skeleton.csv

```

Class Methods

Method	Responsibility
init	Initializes paths, version, options, and ordered model maps.
debug_print, trace_print, error_print	Diagnostic and fatal reporting.
ensure_gl_gen_schema, gl_gen_schema_location	Provide the generic datatype schema.
concept_item_type	Maps LHM datatypes to XBRL item types.
LC3, titleCase, SC, escape_text	Name and text normalization.
getRecord, getParent, getChildren, getElementID	Model lookup.
domainMember, defineHypercube	Build dimensional relationships.
roleRecord, linkPresentation	Build role and presentation structures.
normalize_lhm_record	Adapts UADC and legacy LHM columns.
load_csv_data	Loads records and builds presentation/dimension maps.
process_records	Finalizes repeated-class and role relationships.
generate_taxonomy_files	Writes schemas and linkbases.
json_meta_file	Writes metadata and the optional CSV skeleton.

Validation and Constraints

Run **tests/test_xbrlgl_generator_uadc_lhm.py** and **tests/validate_taxonomy.py**, then validate metadata with Arelle. The OIM entry point is **en16931-oim-2026-07-05.xsd** and the dimensional definition linkbase is **en16931-def-2026-07-05.xml**. Tuple/content entry schemas are not part of this PoC profile.

8.4.14 update_lhm_definitions_from_pdf.py

Role

Updates LHM **definition** values from the Description column of EN 16931-1 Table 2.

Command

```
& $python .\tools\update_lhm_definitions_from_pdf.py STANDARD.pdf LHM.csv
```

[--first-page 43] [--last-page 75]

The LHM is rewritten in place. Page numbers are one-based.

Processing

1. **extract_descriptions** uses **pdfplumber** to inspect tables in the selected page range.
2. **clean_identifier** extracts BG and BT identifiers from the first cell.
3. **clean_description** normalizes whitespace and removes a Description label.
4. Description fragments continuing across table rows are concatenated.
5. **DESCRIPTION_OVERRIDES** supplies curated replacements for extraction gaps.
6. Matching LHM definitions are updated; missing extractions and unresolved empty definitions are reported separately.

Functions and Data

Item	Responsibility
clean_cell, clean_description, clean_identifier	Normalize PDF cells.
extract_descriptions	Extract identifier-to-description mappings.
DESCRIPTION_OVERRIDES	Fill known PDF extraction gaps.
read_rows, write_rows	Read and rewrite the LHM.
main	Applies extracted definitions and reports unresolved rows.

Exit and Dependency

Returns **1** only when at least one BG or BT row still has an empty definition. Requires **pdfplumber**. A description that could not be extracted but already has a non-empty LHM value is reported without causing failure.

8.4.15 update_lhm_syntax_sequence_from_ubl_xsd.py

Role

Populates LHM **syntax_sequence** from the actual child order in extracted OASIS UBL 2.1 XML Schema files.

Command

```
& $python .\tools\update_lhm_syntax_sequence_from_ubl_xsd.py INPUT.csv `
--output OUTPUT.csv `
--schema-root UBL-2.1\xsd `
[--encoding utf-8-sig]
```

The program does not download UBL schemas.

Processing

1. **SchemaIndex.load_directory** recursively loads every XSD.
2. Global elements, named complex types, inline sequences, and extension sequences are indexed by namespace/local-name pairs.
3. XPath predicates and terminal attributes are removed from structural steps.
4. **syntax_sequence_for_xpath** follows the UBL element declarations and records each one-based child position as four digits, beginning with **0000** for the document root.
5. A terminal attribute is appended as **@name**.
6. Rows are sorted by resolved syntax sequence; unresolved rows retain source order after resolved rows.
7. The column is inserted after **sequence** when absent.

Data Structures and Functions

Item	Responsibility
ElementDecl	Stores an element QName and optional type QName.
SchemaIndex	Indexes XSD elements, types, and child sequences.
clean_xpath_step, split_xpath_steps	Parse structural XPath steps.
split_terminal_attribute, xpath_steps	Separate terminal attributes.
step_qname	Resolves UBL prefixes to namespaces.
syntax_sequence_for_xpath	Traverses the schema order for one XPath.
sort_key	Sorts resolved before unresolved records.
update_lhm	Loads schemas, updates all rows, and writes the output.
main	Parses arguments and reports resolved/total counts.

Validation and Constraints

An unresolved XPath receives a blank sequence but does not cause status **1**. Review the resolved count and run **tests/test_ubl_schema_child_order.py**. Store downloaded schema packages under a cache or generated-output directory unless their licensing and repository role explicitly require commitment.

8.5 Recommended Maintenance Workflow

For a source-model change:

1. Update the editable source CSV or curated coverage data.
2. Build or extend the LHM.
3. Normalize semantic paths, class terms, and elements as appropriate.
4. Update syntax order from the UBL schema.
5. Order rows by EN 16931 Table 2.
6. Run the class/element and coverage audits.
7. Generate and validate the taxonomy.
8. Rebuild and validate round-trip artifacts.

For a binding tutorial or prototype:

1. Generate a syntax binding with **build_syntax_binding.py**.
2. Inspect its generated defaults and multiplicities.
3. Exercise it with **tools/tutorial/syntax_binding_sample.py**.
4. Move reviewed binding definitions to **specs/bindings/**.
5. Use the operational **src/** converter and its regression tests for PoC deliverables.

8.6 Validation Matrix

Changed area	Minimum checks
LHM identifiers or hierarchy	test_lhm_semantic_paths.py , test_lhm_hierarchical_csv_layout.py
Class terms or elements	check_lhm_class_element.py
UBL syntax order	test_ubl_schema_child_order.py
Taxonomy generation	test_xbrlgl_generator_uadc_lhm.py , validate_taxonomy.py
Structured CSV or metadata	test_syntax_binding.py , test_xbrl_csv_metadata_arelle.py
Reverse conversion	test_syntax_binding_reverse.py , test_roundtrip_xml_ubl_schema.py
Phase 2 bindings	Target-specific ADS tests and semantic binding tests

8.7 Dependencies

Most tools use only the Python standard library. Additional dependencies are:

Dependency	Used by
pypdf	audit_en16931_coverage.py
pdfplumber	update_lhm_definitions_from_pdf.py
Browser JavaScript APIs	psv_viewer.html
Arele, external validation	Generated taxonomy and metadata validation tests

The taxonomy generator uses local schema resources and does not download them. The round-trip builder invokes repository scripts through the same Python executable used to start the builder.

8.8 Maintenance Rules

- Keep the tool inventory in this document synchronized with **tools/**.
- Update the relevant subsection whenever arguments, functions, generated files, dependencies, or exit behavior change.
- Do not describe tutorial converters as operational runtime converters.
- Record in-place modification and directory-cleaning behavior explicitly.
- Regenerate this document's PDF with VSCode Markdown PDF after Markdown edits.
- Use the configured margins: top and bottom **20mm**, left and right **18mm**.

9. LHM Generation Overview

LHM Generation Documentation

This directory documents how the EN 16931 invoice LHM is generated and reviewed.

Files

- **program_specification.md** - Defines the LHM generation program behavior, input layout, output layout, semantic path rules, element naming rules, and **lhm_level** derivation.
- **user_guide.md** - Gives operator steps for regenerating the LHM CSV, checking output, and troubleshooting common issues.

Related Directories

- **../specs/lhm/source/** - Editable source CSV for controlled LHM changes.
- **../specs/lhm/** - Generated/current LHM CSV used by conversion and taxonomy generation.
- **../tools/** - LHM maintenance scripts, including source rebuild, coverage audit, class/element normalization, and syntax sequence utilities.

The Excel workbook used for human review is local-only and ignored by Git.

10. LHM Generation Program Specification

Program Specification: LHM Generation

1. Purpose

This document specifies the programs used to generate and validate the EN 16931 invoice Logical Hierarchical Model (LHM) CSV for the UADC Proof of Concept.

All paths in this document are relative to the **UADC_PoC** working directory after the repository is pushed or cloned.

2. Scope

The current baseline is the EN 16931-1 invoice semantic model. OpenPeppol BIS Billing is handled later as a CIUS/profile overlay.

Controlled source:

```
specs/lhm/source/EN16931_CIOUS_Invoice_LHM_Source.csv
```

Generated LHM:

```
specs/lhm/EN16931_CIOUS_Invoice_LHM.csv
```

3. Components

3.1 Source-to-LHM builder

Program:

```
tools/build_lhm_from_source.py
```

Responsibilities:

- create an editable source CSV from an existing LHM CSV with **init-source**;
- generate the normalized LHM CSV from the editable source CSV with **build**;
- derive lowerCamelCaseConcatenated semantic path segments from Business Term values;
- derive **class_term** from the nearest parent BG Business Term, singularized;
- derive **lhm_level** for Structured CSV and xBRL-CSV taxonomy modeling;
- generate unique UpperCamelCase **element** values when no override is provided;
- preserve manual override columns.

3.2 Class and element normalizer

Program:

```
tools/normalize_lhm_class_element.py
```

Responsibilities:

- normalize **class_term**;
- normalize **element**;
- ensure generated element values are unique.

Rules:

- BG rows use their own Business Term as **class_term**, singularized.
- BT rows use the nearest parent BG Business Term as **class_term**, singularized.
- **element** is generated from **semantic_path**.
- **element** starts with an uppercase letter.
- If final semantic path segments duplicate, the shortest unique semantic path suffix is used.

3.3 Class and element checker

Program:

```
tools/check_lhm_class_element.py
```

Responsibilities:

- report **class_term** mismatches;
- report **element** mismatches;
- return a non-zero exit code when mismatches are found.

3.4 PDF definition updater

Program:

```
tools/update_lhm_definitions_from_pdf.py
```

Responsibilities:

- extract EN 16931-1 Table 2 Description cells with **pdfplumber**;
- update the LHM **definition** column for extracted BT/BG identifiers;
- apply built-in overrides for rows where PDF extraction is known to split cells;
- report unresolved empty definitions.

This utility updates the specified LHM CSV.

3.5 UBL syntax sequence updater

Program:

```
tools/update_lhm_syntax_sequence_from_ubl_xsd.py
```

Responsibilities:

- read extracted OASIS UBL 2.1 XSD files;
- resolve each LHM XPath against the UBL Invoice schema sequence;
- write **syntax_sequence** values that can be used for XML-order checks;
- keep the downloaded UBL schema package outside version control, normally under **out/cache**.

4. Editable Source CSV

Fields:

```
sequence
syntax_sequence
id
level
type
cardinality
business_term
description
usage_note
req_id
semantic_data_type
path
xpath
semantic_path_override
class_term_override
element_override
label_local
definition_local
source_ref
adjustment_note
```

Important fields:

- **sequence**: EN 16931-1 Table 2 order.
- **syntax_sequence**: UBL Invoice XML schema order, when populated from OASIS UBL 2.1 XSD.
- **id**: EN 16931 identifier such as **BG-4** or **BT-27**.
- **level**: hierarchy level. Invoice is level **0**; **BT-1** is level **1**.
- **cardinality**: source cardinality. Values ending in **..n** are normalized to **..***.

- **business_term**: EN 16931 Business Term.
- **description**: EN 16931 Description. This becomes the LHM **definition**.
- **path**: slash-separated identifier path used to locate the parent BG.
- **xpath**: syntax binding reference path for UBL Invoice where available.
- **semantic_path_override**: optional full semantic path override.
- **class_term_override**: optional class term override.
- **element_override**: optional element name override.

5. Generated LHM CSV

Fields:

```
sequence
syntax_sequence
level
lhm_level
type
identifier
name
datatype
multiplicity
domain_name
definition
module
class_term
id
path
semantic_path
label_local
definition_local
element
xpath
```

Mapping rules:

- **level** preserves the EN 16931/LHM logical hierarchy.
- **lhm_level** is the effective hierarchy used by Structured CSV and xBRL-CSV taxonomy generation.
- **BG-ROOT** has **lhm_level=0**.
- A BG with multiplicity **0..*** or **1..*** has **lhm_level** equal to the nearest ancestor BG with an **lhm_level** plus 1.
- A BG with multiplicity **0..1** or **1..1** has blank **lhm_level**, except **BG-ROOT**.
- A BT has **lhm_level** equal to the nearest ancestor BG with an **lhm_level** plus 1.

- Blank **lhm_level** BG rows are retained in the semantic model but are not emitted as Structured CSV dimensions or xBRL-CSV dimension concepts.
- **name** is copied from **business_term**.
- **syntax_sequence** is copied from the source CSV or populated by the UBL syntax sequence updater.
- **datatype** is copied from **semantic_data_type**.
- **multiplicity** is copied from **cardinality**.
- **definition** is copied from **description**.
- **module** is currently **en16931**.
- **semantic_path** is either **semantic_path_override** or a generated path.
- **element** is either **element_override** or a generated unique UpperCamelCase name.

Semantic path rules:

- Business Terms are converted to lowerCamelCaseConcatenated path segments.
- The semantic path starts at **\$.invoice**.
- **BG-0** is not generated because EN 16931-1 does not define it.

Multiplicity rules:

- LHM **multiplicity** must be one of **0..1**, **0..**, **1..1**, or **1..**
- Source cardinalities **0..n** and **1..n** are normalized to **0..*** and **1..***.
- Other multiplicity values are rejected during LHM generation.

Element name uniqueness rules:

1. Split **semantic_path** into path segments after removing the leading **\$.**
2. Use the final segment as the first candidate.
3. Convert the candidate segment or suffix to UpperCamelCase.
4. If that name is unique, use it as **element**.
5. If it duplicates another row, expand the candidate one segment to the left and try again.
6. Continue until the shortest unique semantic path suffix is found.
7. If no suffix is unique, append the row identifier without hyphens as a final fallback.

Example:

```
$.invoice.precedingInvoiceReference.precedingInvoiceReference
```

The final segment alone would produce **PrecedingInvoiceReference**, which also belongs to the BG row. The generator therefore expands the suffix and produces:

```
PrecedingInvoiceReferencePrecedingInvoiceReference
```

Example:

```
BT-1 Invoice number
semantic_path = $.invoice.invoiceNumber
class_term = Invoice
element = InvoiceNumber
level = 1
```

6. Validation Rules

The LHM checks verify that:

- semantic paths use lowerCamelCaseConcatenated path segments;
- **BG-0** is not present;
- **BG-ROOT** represents Invoice at level **0**;
- **BT-1** is **\$.invoice.invoiceNumber** at level **1**;
- LHM element names are unique;
- multiplicity values are limited to **0..1**, **0..**, **1..1**, and **1..**;
- BG dimension columns are left aligned in hierarchical CSV output;
- non-repeating BGs such as Seller and Buyer are not emitted as dimension columns;
- repeating BGs such as Invoice Line are emitted as dimension columns.

7. Dependencies

Required:

- Python 3.10 or later.

Optional:

- **pdfplumber**, only for **tools/update_lhm_definitions_from_pdf.py**.

8. Non-Goals

The LHM generation programs do not:

- publish generated **out/** files to GitHub;
- fully model OpenPeppol BIS Billing profile constraints;
- validate XBRL taxonomy output.

11. LHM Generation User Guide

User Guide: LHM Generation

1. Working Directory

Run commands from the **UADC_PoC** directory:

```
cd UADC_PoC
```

All paths below are relative to this directory.

Set the Python command for the local Windows environment:

```
$python = 'python'
```

2. Edit the LHM Source

Edit:

```
specs/lhm/source/EN16931_CIOUS_Invoice_LHM_Source.csv
```

Commonly edited columns:

- **description**
- **path**
- **xpath**
- **semantic_path_override**
- **class_term_override**
- **element_override**
- **label_local**
- **definition_local**
- **adjustment_note**

Leave override columns blank when generated values are acceptable.

Use only these multiplicity values in the generated LHM:

```
0..1  
0..*  
1..1  
1..*
```

If source material uses **0..n** or **1..n**, the LHM generator normalizes those values to **0..*** or **1..***.

The generated LHM has both **level** and **lhm_level**:

- **level** preserves the EN 16931/LHM logical hierarchy.
- **lhm_level** is the effective hierarchy for Structured CSV and xBRL-CSV taxonomy generation.
- **0..1** and **1..1** BG rows normally have blank **lhm_level** and are not emitted as dimensions.
- BT rows under those BGs use the nearest ancestor BG with an **lhm_level** plus **1**.
- Repeating BG rows, **0..*** and **1..***, receive their own **lhm_level** and become dimensions.

3. Generate the LHM CSV

Run:

```
& $python .\tools\build_lhm_from_source.py build `
.\specs\lhm\source\EN16931_CIOUS_Invoice_LHM_Source.csv `
.\specs\lhm\EN16931_CIOUS_Invoice_LHM.csv
```

Expected output:

```
Wrote generated LHM CSV: specs\lhm\EN16931_CIOUS_Invoice_LHM.csv
```

4. Optional: Initialize the Source CSV

Use this only when a new editable source CSV must be created from an existing LHM CSV:

```
& $python .\tools\build_lhm_from_source.py init-source `
.\specs\lhm\EN16931_CIOUS_Invoice_LHM.csv `
.\specs\lhm\source\EN16931_CIOUS_Invoice_LHM_Source.csv
```

This command rewrites the source CSV. Use it carefully.

5. Optional: Normalize Class and Element Values

Prefer rebuilding from the source CSV. If the generated LHM CSV has been edited directly, normalize it with:

```
& $python .\tools\normalize_lhm_class_element.py `
.\specs\lhm\EN16931_CIOUS_Invoice_LHM.csv
```

6. Optional: Populate UBL Syntax Sequence

Download and extract the official OASIS UBL 2.1 package into a local, non-versioned directory such as:

```
out/cache/UBL-2.1
```

Then populate **syntax_sequence** values from the UBL Invoice schema:

```
& $python .\tools\update_lhm_syntax_sequence_from_ubl_xsd.py `
.\specs\lhm\EN16931_CIOUS_Invoice_LHM.csv `
--schema-root .\out\cache\UBL-2.1\xsd `
-o .\out\cache\EN16931_CIOUS_Invoice_LHM.syntax_sequence_check.csv
```

Use the generated file to review XML schema order. The EN 16931 **sequence** column remains the semantic Table 2 order; **syntax_sequence** is the UBL XML order used for XML-oriented checks and reverse output ordering.

7. Optional: Update Definitions from PDF

If the EN 16931-1 PDF is available and **pdfplumber** is installed:

```
& $python .\tools\update_lhm_definitions_from_pdf.py `
"<path-to-EN16931-1-pdf>" `
.\specs\lhm\EN16931_CIOUS_Invoice_LHM.csv `
--first-page 43 `
--last-page 75
```

This updates only the CSV file passed on the command line.

8. Check the LHM

Run semantic path checks:

```
& $python .\tests\test_lhm_semantic_paths.py
```

Run class and element checks:

```
& $python .\tools\check_lhm_class_element.py `
.\specs\lhm\EN16931_CIOUS_Invoice_LHM.csv
```

Run hierarchical CSV layout checks:

```
& $python .\tests\test_lhm_hierarchical_csv_layout.py
```

9. Troubleshooting

Duplicate element names

Regenerate the LHM from the source CSV. Element names are generated from **semantic_path** by using the shortest unique suffix. If a duplicate remains intentional, set **element_override** in the source CSV.

Wrong class term

Check the source CSV **path** column. BT rows use the nearest parent BG found in the path.

Wrong semantic path

Set **semantic_path_override** in the source CSV and rebuild the LHM.

Missing PDF descriptions

Check the PDF page range. Some rows may need manual **description** values in the source CSV because PDF table extraction can split cells.

12. Taxonomy Generation Overview

Taxonomy Generation Documentation

This directory documents how the xBRL-CSV taxonomy is generated from the LHM.

Files

- **program_specification.md** - Defines the taxonomy generator inputs, output files, xBRL-CSV restrictions, dimensional relationships, and validation rules.
- **user_guide.md** - Gives command examples for generating and checking the taxonomy with local tests and Arelle.

Related Directories

- **../tools/taxonomy/** - Local UADC-compatible taxonomy generator and GL generic schema template.
- **../specs/lhm/** - LHM CSV used as the taxonomy source.
- **../out/taxonomy/** - Generated taxonomy output tracked by Git as PoC validation evidence.

Phase 1 generates **en16931-oim-.xsd** as the xBRL-CSV taxonomy entry point and **en16931-def.xml** as its dimensional definition linkbase. The entry point references the EN 16931 presentation linkbase so the LHM hierarchy is visible to taxonomy processors. It does not generate **plt-all-.xsd** or **-content-.xsd** tuple/content schemas.

13. Taxonomy Generation Program Specification

Program Specification: Taxonomy Generation

1. Purpose

This document specifies the taxonomy generation program used by the UADC Proof of Concept.

All paths in this document are relative to the repository root after the repository is pushed or cloned. The taxonomy generator used by this PoC is included in this repository.

2. Scope

Input LHM:

```
specs/lhm/EN16931_CIOUS_Invoice_LHM.csv
```

Taxonomy generator:

```
tools/taxonomy/xBRLGL_TaxonomyGenerator.py
```

Output directory:

```
out/taxonomy/
```

The output directory is generated local output and is not intended to be pushed to GitHub.

3. Component

Program:

```
tools/taxonomy/xBRLGL_TaxonomyGenerator.py
```

Responsibilities:

- read the UADC LHM CSV layout through **csv.DictReader**;

- normalize UADC LHM records into the internal XBRL-GL generator layout;
- generate module taxonomy schema files;
- generate an xBRL-CSV dimensional taxonomy schema;
- generate label, presentation, definition linkbase, JSON metadata, and CSV skeleton files.

The generator accepts UADC LHM CSV headers directly.

4. Input Contract

The generator consumes the generated LHM CSV:

```
specs/lhm/EN16931_CIOUS_Invoice_LHM.csv
```

Required fields include:

```
sequence
level
lhm_level
type
name
datatype
multiplicity
definition
module
class_term
id
path
semantic_path
element
xpath
```

Important interpretation rules:

- **module** becomes the module namespace prefix, currently **en16931**.
- **element** becomes the concept name.
- **type** controls whether a row is treated as a field or group for dimensional modeling.
- **multiplicity** is used for dimensional modeling.
- **level**, **path**, and **semantic_path** preserve the logical LHM hierarchy.
- **lhm_level** defines the effective hierarchy for xBRL-CSV taxonomy generation.
- BG rows with blank **lhm_level** are retained as semantic groupings but are not emitted as hypercube, dimension, or primary item concepts.
- BT rows are attached to the cube of the nearest ancestor BG with an **lhm_level**.

Allowed **multiplicity** values are **0..1**, **0..**, **1..1**, and **1..**. The LHM generation step normalizes source values such as **0..n** and **1..n** before taxonomy generation.

5. Taxonomy Output

The EN 16931 PoC output includes:

```
out/taxonomy/en16931/en16931-2026-07-05.xsd
out/taxonomy/en16931/en16931-2026-07-05-presentation.xml
out/taxonomy/en16931/lang/en16931-2026-07-05-label.xml
out/taxonomy/en16931/lang/en16931-2026-07-05-label-ja.xml
out/taxonomy/gen/gl-gen-2026-07-05.xsd
out/taxonomy/plt/en16931-def-2026-07-05.xml
out/taxonomy/plt/en16931-oim-2026-07-05.xsd
```

5.1 Module schema

File pattern:

```
out/taxonomy/<module>/<module>-<version>.xsd
```

For this PoC:

```
out/taxonomy/en16931/en16931-2026-07-05.xsd
```

Responsibilities:

- declare item concepts for LHM Field rows;
- assign **xbrli:periodType="instant"** to **substitutionGroup="xbrli:item"** elements.
- import the GL generic type schema from **../gen/gl-gen-.xsd** when a GL generic type is used.

5.2 Omitted tuple/content schemas

No XBRL 2.1 tuple taxonomy entry point or tuple/content schema is generated for this Structured CSV PoC. In particular, these files are not defined:

- **out/taxonomy/plt/plt-all-.xsd**
- **out/taxonomy/plt/-content-.xsd**

5.3 GL generic type schema

File pattern:

```
out/taxonomy/gen/gl-gen-<version>.xsd
```

Responsibilities:

- provide reusable GL item types such as **gen:amountItemType** and **gen:emailAddressItemType**;
- use the same version namespace as the generated taxonomy;
- be referenced from module schemas as **../gen/gl-gen-.xsd**.

5.4 xBRL-CSV taxonomy schema

File pattern:

```
out/taxonomy/plt/en16931-oim-<version>.xsd
```

Responsibilities:

- define **h_*** hypercube concepts using **xbrldt:hypercubeItem**;
- define **d_*** dimension concepts using **xbrldt:dimensionItem**;
- define **p_*** primary item concepts using **xbrli:item**;
- define the typed dimension domain element **_v**;
- define role types used by the xBRL-CSV definition linkbase;
- reference **en16931-def-.xml**, module labels, and the module presentation linkbase;
- use the EN 16931 target namespace for OIM primary items and dimensions.

Restrictions:

- **en16931-oim** must not define tuple-supporting **complexType**.
- **en16931-oim** must not define **xbrli:tuple** concepts.
- **en16931-oim** must not import the module content schema.

5.5 Definition linkbase

File pattern:

```
out/taxonomy/plt/en16931-def-<version>.xml
```

Responsibilities:

- reference **en16931-oim** role types and dimensional concepts;
- connect primary items, hypercubes, and dimensions;
- represent BG-to-BG hierarchy using dimensional relationships for xBRL-CSV.

The definition linkbase locators for dimensional relationships point to **en16931-oim**.

5.6 Presentation hierarchy

The OIM entry point references **out/taxonomy/en16931/en16931--presentation.xml**. The presentation hierarchy is derived from the LHM parent-child tree. Class/BG rows resolve to the corresponding **p_en16931_*** primary items in **en16931-oim**; BT/fact rows resolve to concepts in the EN 16931 module schema. This makes the LHM hierarchy visible in Arelle's Presentation view without introducing tuple concepts.

6. Validation Rules

The taxonomy regression checks verify that:

- **en16931-oim** contains hypercube, dimension, and primary item definitions;
- **en16931-oim** contains no **xbrli:tuple**;
- **en16931-oim** contains no **complexType**;
- **en16931-oim** does not import **en16931-content**;
- the OIM entry point discovers the presentation linkbase;
- every presentation locator resolves to an OIM primary item or module fact;
- **plt-all** is not generated;
- **en16931-content** is not generated;
- module schemas import **../gen/gl-gen-.xsd**;
- **xbrli:item** elements generated in **en16931-oim** have **xbrli:periodType="instant"**.

7. Dependencies

Required:

- Python 3.10 or later.

8. Non-Goals

The taxonomy generation program does not:

- update LHM source CSV files;
- publish generated **out/** files to GitHub;
- validate the generated DTS with an external XBRL processor;
- validate xBRL-CSV instances beyond the regression checks.

14. Taxonomy Generation User Guide

User Guide: Taxonomy Generation

1. Working Directory

Run commands from the **UADC_PoC** directory:

```
cd UADC_PoC
```

All paths below are relative to this directory. The taxonomy generator is included in **tools/taxonomy/**.

Set the Python command for the local Windows environment:

```
$python = 'python'
```

2. Input

The taxonomy generator uses:

```
specs/lhm/EN16931_CIOUS_Invoice_LHM.csv
```

Regenerate and check the LHM before generating taxonomy when the source model has changed.

3. Generate the Taxonomy

Run the taxonomy regression script:

```
& $python .\tests\test_xbrlgl_generator_uadc_lhm.py
```

This regenerates files under:

```
out/taxonomy/
```

4. Direct Generator Command

To run the taxonomy generator directly:

```
& $python .\tools\taxonomy\xBRLGL_TaxonomyGenerator.py `
.\specs\lhm\EN16931_CIOUS_Invoice_LHM.csv `
-b .\out\taxonomy `
-p en16931 `
-n https://example.com/uada/en16931/invoice/2026-07-05 `
-l ja `
-c JPY
```

Useful options:

- **-b, --base_dir**: output directory.
- **-p, --palette**: palette/module prefix used for generated files.
- **-r, --root**: root element name for JSON metadata and CSV skeleton generation.
- **-l, --lang**: local label language.
- **-c, --currency**: ISO currency for amount facts.
- **-n, --namespace**: namespace. The final 10 characters are used as the version date.
- **-e, --encoding**: input/output encoding. Default is **utf-8-sig**.
- **-t, --trace**: print trace messages.
- **-d, --debug**: print debug messages.

5. Output Files

Key files:

```
out/taxonomy/en16931/en16931-2026-07-05.xsd
out/taxonomy/en16931/en16931-2026-07-05-presentation.xml
out/taxonomy/en16931/lang/en16931-2026-07-05-label.xml
out/taxonomy/en16931/lang/en16931-2026-07-05-label-ja.xml
out/taxonomy/gen/gl-gen-2026-07-05.xsd
out/taxonomy/plt/en16931-def-2026-07-05.xml
out/taxonomy/plt/en16931-oim-2026-07-05.xsd
```

Meaning:

- **en16931/en16931-2026-07-05.xsd**: module item concepts.
- **gen/gl-gen-2026-07-05.xsd**: GL generic item type schema referenced as **../gen/gl-gen-.xsd**.
- **plt/en16931-oim-2026-07-05.xsd**: xBRL-CSV taxonomy entry point with hypercubes, dimensions, primary items, and references to labels, presentation, and definition relationships.
- **plt/en16931-def-2026-07-05.xml**: xBRL-CSV dimensional definition linkbase.
- **en16931/en16931-2026-07-05-presentation.xml**: LHM parent-child hierarchy; BG/Class nodes point to OIM primary items and BT nodes point to module facts.

6. Verify the Taxonomy Separation

Check that the local taxonomy structure is consistent:

```
& $python .\tests\validate_taxonomy.py
```

Expected output:

```
ok: local taxonomy checks passed
```

Check that **en16931-oim** does not contain tuple or content schema definitions:

```
Select-String -Path .\out\taxonomy\plt\en16931-oim-2026-07-05.xsd `
  -Pattern 'en16931-content', 'complexType', 'xbrli:tuple'
```

Expected result: no matches.

Check that **en16931-oim** contains hypercube, dimension, and primary item concepts:

```
Select-String -Path .\out\taxonomy\plt\en16931-oim-2026-07-05.xsd `
  -Pattern
  'xbrldt:hypercubeItem', 'xbrldt:dimensionItem', 'substitutionGroup="xbrli:item"'
```

Check that the module schema imports the GL generic schema:

```
Select-String -Path .\out\taxonomy\en16931\en16931-2026-07-05.xsd `
  -Pattern '../gen/gl-gen-2026-07-05.xsd'
```

Check that **plt-all** and **en16931-content** are not generated:

```
Test-Path .\out\taxonomy\plt\plt-all-2026-07-05.xsd
Test-Path .\out\taxonomy\plt\en16931-content-2026-07-05.xsd
```

Expected result: **False**.

7. Run Arelle Taxonomy Validation

If Arelle is installed, run:

```
& arelleCmdLine.exe `
  --file .\out\taxonomy\plt\en16931-oim-2026-07-05.xsd `
  --validate
```

Expected output includes:

```
[info] validated
```

8. Run the Regression Check

Run:

```
& $python .\tests\test_xbrlgl_generator_uadc_lhm.py
```

Expected output includes:

```
ok: XBRL-GL generator accepted UADC LHM CSV
```

9. Troubleshooting

Python is not found

Use the full Python path:

```
& $python --version
```

en16931-oim contains tuple or content definitions

Regenerate with the current generator:

```
& $python .\tests\test_xbrlgl_generator_uadc_lhm.py
```

The current design is xBRL-CSV only. It keeps hypercube, dimension, and primary item definitions in **en16931-oim**; it does not generate XBRL 2.1 tuple concepts.

The taxonomy uses an unexpected date

Check the **-n** namespace option. The generator uses the final 10 characters of the namespace as the version date.